OPTIMA: Mathematical Foundations and Scheduling Theory for Asymmetric Multiprocessors via Density-Based Sequencing and Online Admissible Preemption Filtering

Risheendra MN
Systems Research
rishindra.hackbox@gmail.com

***Abstract*—Asymmetric multiprocessor architectures, combining high-throughput Performance (P) cores and energy-efficient Efficiency (E) cores with disparate compute capacities, present fundamental challenges for general-purpose operating system schedulers. Existing kernel schedulers rely heavily on empirical heuristics that lack formal mathematical characterization and struggle under mixed latency-critical and compute-intensive workloads. In this paper, we develop the rigorous mathematical foundations and scheduling theory of OPTIMA, an extensible task-scheduling framework implemented via Linux `sched_ext` eBPF.**

We formalize the asymmetric scheduling problem under a dual objective: minimizing total weighted completion time $\sum w_j C_j$ while providing latency-critical prioritization and bounded queuing delay under non-overloaded regimes. We rigorously prove that Weighted Shortest Processing Time (WSPT) sequencing via Smith's Rule is optimal for isolated, single-core non-preemptive queues at time zero ($1||\sum w_j C_j$). We explicitly acknowledge that the generalized online problem with arbitrary release dates, preemption, and asymmetric parallel machines ($R|r_j, \text{pmtn}|\sum w_j C_j$) is strongly NP-hard; accordingly, we cite the classical Kawaguchi-Kyan bound ($\alpha \leq \frac{1+\sqrt{2}}{2} \approx 1.207$) strictly as an identical-machine reference bound and characterize OPTIMA's density-based virtual deadline list scheduling as an online approximation heuristic. Using the Hardy-Littlewood-Pólya Rearrangement Inequality and a continuous fluid progress rate formulation, we prove that matching tasks with higher execution density to Performance cores maximizes the instantaneous priority-weighted progress rate.

Furthermore, we formalize an Online Admissible Preemption Filter (OAPF) based on a dimensionally consistent weighted one-step dispatch delay proxy $\Delta\text{DelayProxy}(c) = w_{\text{curr}}(c) \cdot (\tau_{\text{slice}} + C_{\text{switch}}) - w_{\text{wake}} \cdot R(c)$. We prove that this greedy filter enforces priority inversion safety, eliminates context-switch thrashing, and guarantees strict local one-step dispatch delay reduction. Finally, using a quadratic Lyapunov drift formulation with an exact cross-term expansion, we prove that OPTIMA's adaptive quantum scaling and hierarchical work-stealing maintain strong queue stability ($\limsup \frac{1}{T}\sum \mathbb{E}[q_k(t)] < \infty$) under all strictly admissible workloads ($\sum \lambda_k \bar{p}_k \leq C_{\text{total}} - \epsilon_0$). By bounding virtual deadline advancement ($v_j \leq t + 2\tau_{\max}$ with increments $\Delta v_j \leq \tau_{\max}$), we reconcile strict priority queuing with guaranteed starvation freedom for every admitted task. Complete algorithmic specifications in kernel eBPF hot-paths are provided.

I. System Model and Problem Formulation

A. Heterogeneous Multiprocessor Architecture

We model a heterogeneous asymmetric chip-multiprocessor (AMP) as a set of M discrete computing cores denoted by:

$$\mathcal{M} = \mathcal{P} \cup \mathcal{E}, \quad \mathcal{P} \cap \mathcal{E} = \emptyset \tag{1}$$

where $\mathcal{P} = \{C_1, C_2, \ldots, C_{M_P}\}$ represents the partition of $M_P = |\mathcal{P}|$ Performance cores (P-cores), and $\mathcal{E} = \{C_{M_P+1}, C_{M_P+2}, \ldots, C_{M_P+M_E}\}$ represents the partition of $M_E = |\mathcal{E}|$ Efficiency cores (E-cores), yielding total core count $M = M_P + M_E$.

Each core $C_k \in \mathcal{M}$ is characterized by an intrinsic, time-invariant computational capacity (effective instruction processing frequency) $c_k \in \mathbb{R}_{>0}$, defined as:

$$c_k = \begin{cases} c_P, & \forall C_k \in \mathcal{P} \\ c_E, & \forall C_k \in \mathcal{E} \end{cases} \tag{2}$$

where $c_P > c_E > 0$. We define the asymmetric processing speedup ratio as:

$$\mu \triangleq \frac{c_P}{c_E} > 1 \tag{3}$$

The total aggregate computational capacity rate of the multiprocessor system is:

$$C_{\text{total}} \triangleq \sum_{k=1}^{M} c_k = M_P c_P + M_E c_E \tag{4}$$

The memory and cache subsystem is structured hierarchically: P-cores in $\mathcal{P}$ share a high-bandwidth coherent L3 cache domain (CCX), while E-cores in $\mathcal{E}$ reside in an independent L3 cache domain. Cores may optionally support Simultaneous Multithreading (SMT), where physical core C_k exposes two hardware thread contexts sharing execution units and L1/L2 caches.

B. Dynamic Online Task Model

The workload submitted to the operating system is represented as a dynamic online sequence of tasks:

$$\mathcal{J} = \{J_1, J_2, \ldots, J_n, \ldots\} \tag{5}$$

Each task $J_j \in \mathcal{J}$ is formally specified by a 5-tuple:

$$J_j = (r_j,\, p_j,\, w_j,\, d_j,\, \mathcal{T}_j) \tag{6}$$

where:

1) $r_j \in \mathbb{R}_{\geq 0}$ denotes the release (arrival or wakeup) timestamp of task J_j.
2) $p_j \in \mathbb{R}_{>0}$ represents the nominal processing requirement (computational work volume in reference execution cycles or execution duration measured on a baseline core of capacity c_E). In the kernel implementation, p_j is estimated via an Exponentially Weighted Moving Average (EWMA) of past burst durations.
3) $w_j \in [w_{\min}, w_{\max}] \subset \mathbb{R}_{>0}$ denotes the task priority weight, derived from POSIX nice levels via $w_j = 100 \times 1.25^{-\text{nice}_j}$, with $w_{\min} \geq 10$.
4) $d_j \in \mathbb{R}_{>0} \cup \{\infty\}$ is the target completion deadline. Tasks are categorized into latency-critical/interactive tasks $\mathcal{J}_{\text{RT}} \triangleq \{J_j \in \mathcal{J} \mid d_j < \infty\}$ and compute-bound batch tasks $\mathcal{J}_{\text{Batch}} \triangleq \{J_j \in \mathcal{J} \mid d_j = \infty\}$.
5) $\mathcal{T}_j \in \{\text{Interactive}, \text{Batch}\}$ designates the behavioral execution classification.

Definition 1 (Task Density)**.** *For any task $J_j \in \mathcal{J}$, its task density ρ_j is defined as the ratio of its priority weight to its nominal execution requirement:*

$$\rho_j \triangleq \frac{w_j}{p_j} \tag{7}$$

representing the marginal priority weight delivered per unit of computational work.

C. Execution Capacity Scaling

The physical execution duration of task J_j when dispatched to processor core $C_k \in \mathcal{M}$ depends inversely on the processing capacity c_k:

$$p_{j,k} = \frac{p_j}{c_k} = \begin{cases} \dfrac{p_j}{c_P}, & C_k \in \mathcal{P} \\ \dfrac{p_j}{c_E} = \mu \cdot \dfrac{p_j}{c_P}, & C_k \in \mathcal{E} \end{cases} \tag{8}$$

Thus, executing task J_j on a Performance core accelerates execution by a factor of μ relative to execution on an Efficiency core.

D. Problem Formulation and Real-Time Characterization

Let $S_j \geq r_j$ and C_j denote the scheduling start time and final completion time of task J_j under schedule $\mathcal{S}$, respectively. If task J_j undergoes preemptive execution across time slots, its total completed work satisfies:

$$\sum_{k \in \mathcal{M}} c_k \int_{S_j}^{C_j} \mathbf{1}_{\{J_j \text{ runs on } C_k \text{ at time } t\}}\, dt = p_j \tag{9}$$

The primary objective is to minimize the total weighted completion time across all tasks:

$$\min_{\mathcal{S}} \quad \Phi(\mathcal{S}) \triangleq \sum_{J_j \in \mathcal{J}} w_j C_j(\mathcal{S}) \tag{10}$$

$$\text{subject to} \quad S_j \geq r_j, \quad \forall J_j \in \mathcal{J} \tag{11}$$

$$\sum_{J_j \in \mathcal{J}} \mathbf{1}_{\{J_j \text{ on } C_k \text{ at } t\}} \leq 1, \quad \forall C_k \in \mathcal{M},\ t \geq 0 \tag{12}$$

Constraint (11) enforces causality, and Constraint (12) guarantees non-overlapping single-task execution per physical processing core.

Remark 1 (Real-Time Guarantees and Admission Control)**.** *In an open, general-purpose operating system without hard admission control, arbitrary processes may arrive concurrently without reservation contracts. Under arbitrary adversarial overload ($\sum_{j \in \mathcal{J}_{\text{RT}}} p_j/(d_j - r_j) > M_P$), it is mathematically impossible for any online scheduler to guarantee hard real-time deadlines ($C_j \leq d_j$) for all tasks. Consequently, OPTIMA formulates real-time support as:*

1) ***Latency-Critical Prioritization:*** *Latency-critical tasks are prioritized via zero-lag local dispatch and minimal virtual deadlines ($v_j = t$), ensuring immediate dispatch to Performance cores when capacity is available.*
2) ***Bounded Queuing Delay under Non-Overload:*** *Under non-overloaded regimes where the latency-critical workload utilization satisfies $\sum_{j \in \mathcal{J}_{\text{RT}}} \frac{p_j}{d_j - r_j} \leq U_{\text{RT}} < M_P$, OPTIMA provides bounded queuing delay and rapid dispatch onto Performance cores without priority inversion.*
3) ***Graceful Degradation:*** *Under system overload, OPTIMA provides graceful degradation by minimizing total weighted completion time $\sum w_j C_j$, preventing unbounded starvation.*

II. WSPT Scheduling: Single-Core Optimality, Computational Complexity, and Asymmetric Partitioning

A. Smith's Rule Optimality on Single-Core Dedicated Queues

We begin by establishing the formal scope within which Weighted Shortest Processing Time (WSPT) sequencing is strictly optimal: an isolated single-core queue with simultaneously available independent jobs.

Lemma 1 (Optimality of Single-Core Smith's Rule $(1||\sum w_j C_j)$)**.** *Let $\mathcal{J}_k = \{J_1, J_2, \ldots, J_n\}$ be a set of n independent tasks available simultaneously at time t_0 ($r_j = t_0, \forall j$) on a single isolated processor core C_k with compute capacity c_k. Any non-preemptive schedule that orders tasks in non-increasing order of task density:*

$$\rho_{(1)} \geq \rho_{(2)} \geq \cdots \geq \rho_{(n)} \tag{13}$$

where $\rho_j = w_j/p_j$, strictly minimizes the total weighted completion time $\sum_{j=1}^{n} w_j C_j$.

Proof. We proceed via an adjacent pairwise interchange argument. Assume for contradiction that there exists an optimal

schedule $\mathcal{S}^*$ that does not satisfy the non-increasing density ordering (13).

Since $\mathcal{S}^*$ does not follow the monotonic density ordering everywhere, there must exist at least two adjacent tasks, say J_a immediately followed by J_b, such that:

$$\rho_a < \rho_b \iff \frac{w_a}{p_a} < \frac{w_b}{p_b} \iff w_a p_b < w_b p_a \tag{14}$$

Let $t \geq t_0$ denote the start time of task J_a under schedule $\mathcal{S}^*$. Under $\mathcal{S}^*$, task J_a executes during $[t,\, t + p_{a,k})$, and task J_b executes during $[t + p_{a,k},\, t + p_{a,k} + p_{b,k})$. Their respective completion times are:

$$C_a(\mathcal{S}^*) = t + \frac{p_a}{c_k} \tag{15}$$

$$C_b(\mathcal{S}^*) = t + \frac{p_a}{c_k} + \frac{p_b}{c_k} \tag{16}$$

The combined contribution of J_a and J_b to the objective $\Phi(\mathcal{S}^*)$ is:

$$\begin{aligned}\Phi_{a,b}(\mathcal{S}^*) &= w_a C_a(\mathcal{S}^*) + w_b C_b(\mathcal{S}^*) \\ &= w_a \left(t + \frac{p_a}{c_k} \right) + w_b \left(t + \frac{p_a + p_b}{c_k} \right) \\ &= (w_a + w_b)t + \frac{1}{c_k}\Big(w_a p_a + w_b p_a + w_b p_b \Big)\end{aligned} \tag{17}$$

Now construct an alternative schedule $\mathcal{S}'$ by interchanging the execution positions of J_a and J_b, while preserving the execution positions of all tasks scheduled prior to time t and all tasks scheduled after time $t + \frac{p_a + p_b}{c_k}$.

Under schedule $\mathcal{S}'$, task J_b executes first during $[t,\, t + p_{b,k})$, followed by task J_a during $[t + p_{b,k},\, t + p_{b,k} + p_{a,k})$. Their completion times under $\mathcal{S}'$ become:

$$C_b(\mathcal{S}') = t + \frac{p_b}{c_k} \tag{18}$$

$$C_a(\mathcal{S}') = t + \frac{p_b}{c_k} + \frac{p_a}{c_k} \tag{19}$$

The combined contribution of J_a and J_b under $\mathcal{S}'$ is:

$$\begin{aligned}\Phi_{a,b}(\mathcal{S}') &= w_b C_b(\mathcal{S}') + w_a C_a(\mathcal{S}') \\ &= w_b \left(t + \frac{p_b}{c_k} \right) + w_a \left(t + \frac{p_b + p_a}{c_k} \right) \\ &= (w_a + w_b)t + \frac{1}{c_k}\Big(w_b p_b + w_a p_b + w_a p_a \Big)\end{aligned} \tag{20}$$

For all tasks $J_j \notin \{J_a, J_b\}$, completion times are unaffected:

1) Tasks completed before time t finish before the interchange occurs, so $C_j(\mathcal{S}') = C_j(\mathcal{S}^*)$.
2) Tasks scheduled after J_a and J_b begin at $t + \frac{p_b + p_a}{c_k} = t + \frac{p_a + p_b}{c_k}$. Because the total execution duration of $\{J_a, J_b\}$ is invariant under permutation, the start and completion times of all subsequent tasks remain identical: $C_j(\mathcal{S}') = C_j(\mathcal{S}^*)$ for all subsequent j.

Now evaluate the cost difference $\Delta\Phi \triangleq \Phi(\mathcal{S}') - \Phi(\mathcal{S}^*)$:

$$\begin{aligned}\Delta\Phi &= \Phi_{a,b}(\mathcal{S}') - \Phi_{a,b}(\mathcal{S}^*) \\ &= \frac{1}{c_k}\Big(w_b p_b + w_a p_b + w_a p_a - w_a p_a - w_b p_a - w_b p_b \Big) \\ &= \frac{1}{c_k}\Big(w_a p_b - w_b p_a \Big)\end{aligned} \tag{21}$$

From the inversion condition (14), $w_a p_b - w_b p_a < 0$. Since $c_k > 0$, it follows that:

$$\Delta\Phi = \frac{1}{c_k}(w_a p_b - w_b p_a) < 0 \implies \Phi(\mathcal{S}') < \Phi(\mathcal{S}^*) \tag{22}$$

This contradicts the assumption that $\mathcal{S}^*$ is optimal. Since the permutation group S_n is finite and generated by adjacent transpositions, any schedule can be transformed into non-increasing density order via a finite sequence of adjacent exchanges, with each exchange strictly decreasing the objective function until no inversions remain. Thus, Smith's density ordering is uniquely optimal for single-core non-preemptive queues at time zero. □

B. Computational Complexity of Multiprocessor Scheduling and NP-Hardness

While Smith's Rule provides an exact, polynomial-time solution for the static single-machine problem $1||\sum w_j C_j$, the generalized operating system scheduling problem on asymmetric multiprocessors is computationally intractable.

Using standard Graham notation $\alpha|\beta|\gamma$:

1) **Release Dates on a Single Core ($1|r_j|\sum w_j C_j$):** Introducing dynamic task release dates makes the single-core non-preemptive problem strongly NP-hard, as proven by Lenstra, Rinnooy Kan, and Brucker (1977).
2) **Preemptive Scheduling with Release Dates ($1|r_j, \textbf{pmtn}|\sum w_j C_j$):** Even when arbitrary preemption is permitted, minimizing total weighted completion time with dynamic arrivals remains strongly NP-hard, as proven by Lenstra, Rinnooy Kan, and Brucker (1977). (Bruno, Coffman, and Sethi, 1974, established NP-completeness for precedence-constrained task graphs on identical processors).
3) **Parallel Machines ($P||\sum w_j C_j$):** Scheduling independent jobs without release dates on identical parallel machines is strongly NP-hard (Garey and Johnson, 1979).
4) **Asymmetric / Unrelated Parallel Machines ($R|r_j, \textbf{pmtn}|\sum w_j C_j$):** Scheduling dynamic tasks on asymmetric machines with disparate core capacities ($c_P \neq c_E$), arbitrary release dates, and preemption is strongly NP-hard.

Proposition 1 (Computational Intractability of Global Multi-Core Minimization)**.** *Finding a schedule that globally minimizes total weighted completion time $\sum w_j C_j$ on an asymmetric multiprocessor $\mathcal{M} = \mathcal{P} \cup \mathcal{E}$ with online release dates and preemption ($R|r_j, pmtn|\sum w_j C_j$) is strongly NP-hard. No online deterministic scheduler can guarantee an exact global minimum in polynomial time unless* $\mathrm{P} = \mathrm{NP}$.

C. Approximation Ratio Reference and Online List Scheduling

In classical scheduling theory, Kawaguchi and Kyan (1986) analyzed the worst-case approximation performance of WSPT list scheduling:

Theorem 1 (Kawaguchi-Kyan Reference Bound on Identical Machines)**.** *For the offline scheduling of independent jobs on M identical parallel processors without release dates ($P||\sum w_jC_j$), the WSPT list scheduling heuristic achieves an approximation ratio bounded by:*

$$\frac{\Phi(\mathcal{S}_{\mathrm{WSPT}})}{\Phi(\mathcal{S}_{\mathrm{OPT}})} \leq \frac{1+\sqrt{2}}{2} \approx 1.207 \tag{23}$$

Remark 2 (Scope and Application to Asymmetric Architectures)**.** *The Kawaguchi-Kyan bound $\alpha \leq 1.207$ strictly applies to* identical parallel machines *(P) with simultaneous job availability at time zero. On asymmetric multiprocessors (Q or R) with dynamic release dates r_j, the Kawaguchi-Kyan bound does not apply directly.*

Consequently, OPTIMA does not *claim global multi-core optimality via Smith's Rule, nor does it claim that the 1.207 bound holds unconditionally across asymmetric cores. Instead, OPTIMA employs density-based list scheduling and virtual deadline ordering as an* ***online approximation heuristic*** *inspired by the single-core optimality of Smith's Rule and the classical identical-machine bound.*

D. Instantaneous Backlog Weight Depletion via the Rearrangement Inequality

We now provide the theoretical justification for density-tiered assignment across asymmetric cores $\mathcal{M} = \mathcal{P} \cup \mathcal{E}$ with $c_P > c_E$.

Theorem 2 (Maximization of Instantaneous Backlog Weight Depletion Rate)**.** *Let $\mathcal{M} = \mathcal{P} \cup \mathcal{E}$ be an asymmetric multiprocessor system with capacities $c_P > c_E$. At any instant t, let $\mathcal{Q}(t)$ denote the set of active uncompleted tasks, and let $W(t) \triangleq \sum_{J_j \in \mathcal{Q}(t)} w_j$ denote total remaining priority weight.*

Matching tasks with higher density $\rho_j = w_j/p_j$ to Performance cores strictly maximizes the instantaneous rate of remaining weight reduction $-\dot{W}(t)$.

Proof. Using the integral representation of total weighted completion time:

$$\sum_{J_j \in \mathcal{J}} w_j C_j = \int_0^\infty W(t)\, dt \tag{24}$$

To heuristically minimize $\int_0^\infty W(t)\, dt$, a greedy online scheduler should maximize the instantaneous rate of weight decay $-\dot{W}(t)$ at every time t.

At any time t, let $J_{\sigma(k)}$ be the task currently executing on core $C_k \in \mathcal{M}$. The computational rate delivered to task $J_{\sigma(k)}$ is $c_k/p_{\sigma(k)}$ per second. Because task $J_{\sigma(k)}$ completes after executing $p_{\sigma(k)}$ work, the effective rate of weight retirement delivered by core C_k is:

$$-\dot{W}_k(t) = w_{\sigma(k)} \cdot \frac{c_k}{p_{\sigma(k)}} = \rho_{\sigma(k)} \cdot c_k \tag{25}$$

Summing over all M cores, the total instantaneous rate of weight depletion is:

$$-\dot{W}(t) = \sum_{k=1}^{M} \rho_{\sigma(k)} c_k = \sum_{k \in \mathcal{P}} \rho_{\sigma(k)} c_P + \sum_{k \in \mathcal{E}} \rho_{\sigma(k)} c_E \tag{26}$$

Now consider any two tasks J_1 and J_2 running concurrently, with J_1 assigned to a P-core (c_P) and J_2 assigned to an E-core (c_E). Suppose for contradiction that $\rho_1 < \rho_2$. The weight depletion rate under this assignment $\mathcal{A}$ is:

$$R(\mathcal{A}) = \rho_1 c_P + \rho_2 c_E \tag{27}$$

Interchanging the core assignments of J_1 and J_2 produces assignment $\mathcal{A}'$ with depletion rate:

$$R(\mathcal{A}') = \rho_2 c_P + \rho_1 c_E \tag{28}$$

Computing the difference:

$$\begin{aligned} R(\mathcal{A}') - R(\mathcal{A}) &= (\rho_2 c_P + \rho_1 c_E) - (\rho_1 c_P + \rho_2 c_E) \\ &= (\rho_2 - \rho_1)(c_P - c_E) \end{aligned} \tag{29}$$

Because $\rho_2 > \rho_1$ and $c_P > c_E$, we have $(\rho_2 - \rho_1)(c_P - c_E) > 0$, which implies $R(\mathcal{A}') > R(\mathcal{A})$.

More generally, by the Hardy-Littlewood-Pólya Rearrangement Inequality, for any two real sequences sorted in non-increasing order:

$$\rho_{(1)} \geq \rho_{(2)} \geq \cdots \geq \rho_{(M)} \tag{30}$$

$$c_{(1)} \geq c_{(2)} \geq \cdots \geq c_{(M)} \tag{31}$$

the inner product $\sum_{k=1}^{M} \rho_{\sigma(k)} c_k$ is strictly maximized if and only if the permutation σ matches the monotonic ordering, i.e., $\sigma(k) = (k)$. Because $c_k = c_P$ for all $k \in \mathcal{P}$ and $c_k = c_E$ for all $k \in \mathcal{E}$ with $c_P > c_E$, maximum weight depletion requires:

$$\min_{k \in \mathcal{P}} \rho_{\sigma(k)} \geq \max_{k' \in \mathcal{E}} \rho_{\sigma(k')} \tag{32}$$

Therefore, allocating tasks with highest densities to Performance cores strictly maximizes the instantaneous rate of system backlog reduction. □

Remark 3 (Instantaneous vs Global Optimality)**.** *Maximizing $-\dot{W}(t)$ is an instantaneous (greedy) rate maximization at time t. Because future arrivals $r_j > t$ dynamically alter queue state, instantaneous rate maximization does not constitute a proof of global multi-stage offline optimality over $[0, \infty)$, but provides a principled mathematical basis for asymmetric core partitioning.*

E. Density-Guided Domain Placement via Continuous Fluid Progress Rate

When tasks wake up dynamically, full combinatorial reassignment across all cores is computationally prohibitive. OPTIMA formulates core allocation between Performance and Efficiency domains through a continuous fluid progress rate relaxation.

Under a fluid relaxation where active jobs make continuous progress, the instantaneous priority-weighted rate of progress across the multiprocessor is:

$$\mathcal{R}(t) \triangleq \sum_{k=1}^{M} \rho_{\sigma(k)} c_k = \sum_{k \in \mathcal{P}} \rho_{\sigma(k)} c_P + \sum_{k \in \mathcal{E}} \rho_{\sigma(k)} c_E \tag{33}$$

where $\rho_{\sigma(k)} = w_{\sigma(k)}/p_{\text{est},\sigma(k)}$ is the execution density of the task executing on core k. By the Hardy-Littlewood-Pólya Rearrangement Inequality (Theorem 2), $\mathcal{R}(t)$ is strictly maximized when tasks with higher execution density are assigned to faster Performance cores ($c_P > c_E$).

OPTIMA operationalizes this fluid principle directly in kernel eBPF as an $\mathcal{O}(1)$ domain placement policy:

- High-priority latency-critical tasks ($w_j \geq w_{\text{RT}}$) are unconditionally placed into the dedicated Performance queue (`DSQ_PERF`).
- Long-running compute batch tasks ($p_{\text{est},j} \geq 3\,\text{ms}$ or tagged as batch) have low execution density ρ_j and are placed into the Efficiency domain queue (`DSQ_EFF`), offloading sustained execution from P-cores.
- Short interactive tasks ($p_{\text{est},j} < 3\,\text{ms}$) with high execution density are routed to the domestic shared Performance queue (`DSQ_SHARED_P`), preserving Performance core bandwidth for rapid interactive turnaround.

This eliminates uncoordinated thread racing while providing a deterministic $\mathcal{O}(1)$ kernel hot-path decision rule grounded in fluid rate maximization.

III. Online Admissible Preemption Filtering (OAPF)

A. The Online Preemption Decision Dilemma

In an asymmetric multiprocessing system, when a high-priority interactive or latency-critical task $J_{\text{wake}} \in \mathcal{J}_{\text{RT}}$ arrives at time t, all Performance cores $\mathcal{P}$ may be actively executing batch tasks. The scheduler faces an immediate online dilemma:

- **Option 1 (Non-Preemptive Wait):** Enqueue J_{wake} and allow the running task to complete its allocated time quantum, avoiding context-switch overhead but incurring queue latency for J_{wake}.
- **Option 2 (Preemptive Displacement):** Select an active P-core $c^* \in \mathcal{P}$, immediately preempt its currently running task $J_{\text{curr}}(c^*)$, and schedule J_{wake} on core c^*.

Preemption is not cost-free: preempting a task introduces a context-switch and cache displacement overhead $C_{\text{switch}} > 0$, while delaying the running task J_{curr} by the execution duration of J_{wake} plus switching overhead.

Remark 4 (Nature of the Algorithmic Decision)**.** *OPTIMA does* not *explore an exponential, multi-stage Branch-and-Bound search tree over future schedules. Instead, OPTIMA implements an* ***Online Admissible Preemption Filter (OAPF) / Greedy Decision Rule*** *that evaluates candidate Performance cores $c \in \mathcal{P}$ in $\mathcal{O}(M_P)$ bounded kernel time upon each task wakeup.*

B. Dimensionally Consistent Cost Formulation

For each candidate Performance core $c \in \mathcal{P}$, let $J_{\text{curr}}(c)$ denote the task executing on core c. Let:

- $w_{\text{curr}}(c)$ be the priority weight of running task $J_{\text{curr}}(c)$.
- $t_{\text{start}}(c)$ be the start timestamp of $J_{\text{curr}}(c)$'s current time slice.
- $\tau(c)$ be the allocated quantum of $J_{\text{curr}}(c)$.
- $R(c) \triangleq \max\Big(0,\, \tau(c) - \big(t - t_{\text{start}}(c)\big)\Big)$ be the remaining quantum duration of $J_{\text{curr}}(c)$ in time units.
- w_{wake} be the priority weight of waking task J_{wake}.
- p_{wake} be the nominal execution work / quantum assigned to J_{wake} (in kernel: bounded by $\tau_{\min}$).
- C_{switch} be the context-switch and cache invalidation penalty in time units.

Definition 2 (Weighted One-Step Dispatch Delay Proxy)**.** *Consider candidate core $c \in \mathcal{P}$ currently executing task $J_{\text{curr}}(c)$ with remaining quantum $R(c) > 0$. Let waking task J_{wake} have weight w_{wake} and allocated time slice $\tau_{\text{slice}} = \tau_{\min}$. If the scheduler chooses* ***Option 1 (Wait)****, waking task J_{wake} is delayed by $R(c)$, incurring weighted dispatch delay:*

$$\text{Delay}_{\text{wait}}(c) = w_{\text{wake}} \cdot R(c) \tag{34}$$

If the scheduler chooses ***Option 2 (Preempt)****, J_{wake} executes immediately with 0 delay. However, running task $J_{\text{curr}}(c)$ is suspended during J_{wake}'s slice (τ_{slice} time units) plus context-switch overhead C_{switch}, incurring weighted delay penalty:*

$$\text{Delay}_{\text{preempt}}(c) = w_{\text{curr}}(c) \cdot (\tau_{\text{slice}} + C_{\text{switch}}) \tag{35}$$

The net differential in one-step weighted dispatch delay is:

$$\begin{aligned}\Delta\text{DelayProxy}(c) &\triangleq \text{Delay}_{\text{preempt}}(c) - \text{Delay}_{\text{wait}}(c) \\ &= w_{\text{curr}}(c) \cdot (\tau_{\text{slice}} + C_{\text{switch}}) - w_{\text{wake}} \cdot R(c)\end{aligned} \tag{36}$$

Remark 5 (Dimensional Consistency and Overhead Grounding)**.** *In* (36)*, both terms possess unified physical dimensions of* [weight $\times$ time] *(priority-weighted delay units):*

- $w_{\text{curr}}(c) \cdot (\tau_{\text{slice}} + C_{\text{switch}})$ *has units* $[w] \times [t]$.
- $w_{\text{wake}} \cdot R(c)$ *has units* $[w] \times [t]$.

C_{switch} is nominally set to $50\,\mu$s (with an effective operational sensitivity range of 20–100 μs), reflecting microarchitectural cache-line reload and TLB refill penalties on modern superscalar cores.

C. Online Admissible Pruning Invariants

OPTIMA evaluates $\Delta\text{DelayProxy}(c)$ across candidate cores using four pruning invariants:

1) **Invariant 1 (Idle Core Trivial Minimum):** If candidate core c is idle ($w_{\text{curr}}(c) = 0$), then $\text{Delay}_{\text{preempt}}(c) = 0$ (no task is displaced). Thus:

$$\Delta\text{DelayProxy}(c) = -w_{\text{wake}} \cdot R(c) < 0 \tag{37}$$

Core c is immediately selected for dispatch without preemption.

2) **Invariant 2 (Priority Inversion Safety):** If $w_{\text{curr}}(c) \geq w_{\text{wake}}$, preempting core c would displace an equal- or higher-priority running task to execute a lower-priority waking task, causing priority inversion. Furthermore, since $\tau_{\text{slice}} + C_{\text{switch}} \geq R(c)$ in typical quantum regimes, $\Delta\text{DelayProxy}(c) \geq 0$. The candidate is strictly inadmissible and pruned.
3) **Invariant 3 (Granularity / Anti-Thrashing Threshold):** Preempting core c yields a net delay reduction ($\Delta\text{DelayProxy}(c) < 0$) if and only if:

$$R(c) > \delta_{\min}(c) \triangleq \frac{w_{\text{curr}}(c) \cdot (\tau_{\text{slice}} + C_{\text{switch}})}{w_{\text{wake}}} \quad (38)$$

If $R(c) \leq \delta_{\min}(c)$, the execution gain is insufficient to amortize displacement and context-switch penalties. The candidate is safely pruned.
4) **Invariant 4 (Incumbent Greedy Cutoff):** Maintain the best incumbent delay reduction $\Delta\text{Proxy}^* \triangleq \min_{j \in \text{visited}} \Delta\text{DelayProxy}(j)$, initialized to 0. If $\Delta\text{DelayProxy}(c) \geq \Delta\text{Proxy}^*$, candidate core c cannot improve upon the best discovered candidate. The candidate is pruned.

D. Local One-Step Dispatch Delay Reduction

Theorem 3 (Local One-Step Dispatch Delay Reduction under Admissible Preemption)**.** *Let $\mathcal{P}_{\text{admissible}} \subseteq \mathcal{P}$ be the set of candidate Performance cores satisfying:*

1) *Priority superiority: $w_{\text{wake}} > w_{\text{curr}}(c)$;*
2) *Anti-thrashing remaining runtime: $R(c) > \delta_{\min}(c)$, where*

$$\delta_{\min}(c) \triangleq \frac{w_{\text{curr}}(c) \cdot (\tau_{\text{slice}} + C_{\text{switch}})}{w_{\text{wake}}} \quad (39)$$

If $\mathcal{P}_{\text{admissible}} \neq \emptyset$, selecting core $c^* = \arg\min_{c \in \mathcal{P}_{\text{admissible}}} \Delta\text{DelayProxy}(c)$ and preempting $J_{\text{curr}}(c^*)$ guarantees that the one-step weighted dispatch delay is strictly reduced:

$$\Delta\text{DelayProxy}(c^*) < 0 \quad (40)$$

Conversely, if $\mathcal{P}_{\text{admissible}} = \emptyset$ or candidate cores satisfy $R(c) \leq \delta_{\min}(c)$, preemption is suppressed, bounding priority inversion to at most $\tau(c) \leq \tau_{\max}$.

Proof. Under non-preemptive wait, J_{wake} incurs waiting delay $R(c^*)$, while $J_{\text{curr}}(c^*)$ incurs 0 delay, yielding weighted delay $\text{Delay}_{\text{wait}} = w_{\text{wake}} R(c^*)$. Under preemption, J_{wake} incurs 0 dispatch delay, while $J_{\text{curr}}(c^*)$ incurs delay $(\tau_{\text{slice}} + C_{\text{switch}})$, yielding weighted delay $\text{Delay}_{\text{preempt}} = w_{\text{curr}}(c^*)(\tau_{\text{slice}} + C_{\text{switch}})$.

Computing $\Delta\text{DelayProxy}(c^*) = \text{Delay}_{\text{preempt}} - \text{Delay}_{\text{wait}}$:

$$\Delta\text{DelayProxy}(c^*) = w_{\text{curr}}(c^*)(\tau_{\text{slice}} + C_{\text{switch}}) - w_{\text{wake}} R(c^*) \quad (41)$$

Since $c^* \in \mathcal{P}_{\text{admissible}}$, $R(c^*) > \delta_{\min}(c^*)$, which implies $w_{\text{wake}} R(c^*) > w_{\text{curr}}(c^*)(\tau_{\text{slice}} + C_{\text{switch}})$, ensuring $\Delta\text{DelayProxy}(c^*) < 0$.

When preemption is suppressed (e.g., $w_{\text{curr}}(c) \geq w_{\text{wake}}$ or $R(c) \leq \delta_{\min}(c)$), the waking task waits for quantum expiration. Because each executing slice is clamped to $\tau(c) \leq \tau_{\max}$, the maximum waiting delay is bounded by $\tau_{\max}$, guaranteeing bounded priority inversion while protecting cache locality and pipeline execution. □

Remark 6 (Tunable Anti-Thrashing Dampening Penalty $C_{\text{switch}} = 50\,\mu$s)**.** *While raw CPU register context switches require only 1–2 μs and individual cache line reloads require tens to hundreds of nanoseconds, repeatedly displacing and reloading a memory-intensive working set introduces cumulative pipeline stalls across thousands of cycles. In OPTIMA, C_{switch} serves strictly as a* tunable anti-thrashing dampening penalty *nominally set to $50\,\mu$s (with an effective operational sensitivity range of 20–100 μs). Rather than modeling a single hardware cache miss, C_{switch} enforces an algorithmic hysteresis barrier, ensuring that preemption is triggered only when the waking task's priority advantage is sufficiently pronounced to justify evicting the incumbent thread and disrupting pipeline throughput.*

E. Pseudocode for Online Admissible Preemption Filter

Algorithm 1 presents the complete, verified logic of the Online Admissible Preemption Filter.

IV. Virtual Deadline Calculus and Queue Stability via Lyapunov Drift

A. Virtual Deadline Calculus

To maintain density-ordered scheduling across dynamic online arrivals without global queue sorting overheads, OPTIMA maintains virtual deadlines in eBPF Dispatch Queues (DSQs).

Definition 3 (Virtual Deadline Update Equation)**.** *When task J_j is enqueued into dispatch queue k at time t, its virtual deadline $v_j(t)$ is computed via:*

$$v_j(t) = \max\left(v_j(t^-),\, t\right) + \Delta v_j(t) \quad (42)$$

where $\Delta v_j(t)$ is the execution-density-scaled virtual deadline advance:

$$\Delta v_j(t) = \text{clamp}\left(\tau_j(t) \cdot \frac{\rho_{\text{ref}}}{\rho_j},\ \tau_{\min},\ \tau_{\max}\right) \quad (43)$$

with task density $\rho_j = \frac{\max(w_j, 10)}{p_{\text{est},j}}$ and reference density $\rho_{\text{ref}} = \frac{w_{\text{ref}}}{p_{\text{ref}}} = \frac{100}{1000\,\mu s} = 0.1\,\mu s^{-1}$. In integer arithmetic:

$$\Delta v_j(t) = \text{clamp}\left(\frac{\tau_j(t) \cdot p_{\text{est},j}}{10 \cdot \max(w_j, 10)},\ \tau_{\min},\ \tau_{\max}\right) \quad (44)$$

Here $\tau_j(t)$ is the adaptive scheduling quantum computed as:

$$\tau_j(t) = \text{clamp}\left(\frac{Q_{\text{base}}}{1 + q_k(t)},\ \tau_{\min},\ \tau_{\max}\right) \quad (45)$$

where $q_k(t)$ denotes current queue depth of target queue k, Q_{base} is base nominal quantum, and $\tau_{\min}, \tau_{\max}$ are hardware enforcement boundaries.

Algorithm 1 OPTIMA-OAPF (Online Admissible Preemption Filter)

Require: Waking task J_{wake}, task context $tctx$, candidate core set $\mathcal{P}$
Ensure: Target preemption CPU c^* (or -1 if wait is optimal)

$c^* \leftarrow -1$
$\Delta\text{Proxy}^* \leftarrow 0$ {Incumbent threshold: must yield net delay reduction}
$now \leftarrow$ scx_bpf_now()
$w_{\text{wake}} \leftarrow \max(tctx.weight, 100)$
$\tau_{\text{slice}} \leftarrow \tau_{\min}$ {Execution slice allocated to waking task}
$C_{\text{switch}} \leftarrow 50\,\mu$s {Context-switch and cache displacement overhead}
for each core $c \in \mathcal{P}$ **do**
  if $c \notin J_{\text{wake}}$.cpus_allowed **then**
    continue
  end if
  $st \leftarrow$ lookup_cpu_run_state(c)
  if $st ==$ NULL **or** $st.pid == 0$ **then**
    return c {Rule 1: Core is idle; trivial zero-cost dispatch}
  end if
  if $st.weight \geq w_{\text{wake}}$ **then**
    continue {Rule 2: Inversion safety; candidate pruned}
  end if
  $elapsed \leftarrow now - st.start_time$
  $R(c) \leftarrow (st.slice > elapsed)\ ?\ (st.slice - elapsed) : 0$
  $\delta_{\min} \leftarrow \dfrac{st.weight \cdot (\tau_{\text{slice}} + C_{\text{switch}})}{w_{\text{wake}}}$
  if $R(c) \leq \delta_{\min}$ **then**
    continue {Rule 3: Anti-thrashing; candidate pruned}
  end if
  $\Delta\text{DelayProxy} \leftarrow st.weight \cdot (\tau_{\text{slice}} + C_{\text{switch}}) - w_{\text{wake}} \cdot R(c)$
  if $\Delta\text{DelayProxy} < \Delta\text{Proxy}^*$ **then**
    $\Delta\text{Proxy}^* \leftarrow \Delta\text{DelayProxy}$
    $c^* \leftarrow c$
  end if
end for
if $c^* \geq 0$ **and** $\Delta\text{Proxy}^* < 0$ **then**
  scx_bpf_kick_cpu(c^*, SCX_KICK_PREEMPT)
  return c^*
end if
return -1 {Wait for quantum expiration is optimal}

Remark 7 (Decoupling Domain Placement from Intra-Queue Virtual-Deadline Ordering)**.** *We explicitly decouple coarse-grained domain placement from fine-grained intra-queue scheduling:*

1) ***Domain Placement (Algorithm 3):*** *Tasks are routed to target DSQs using simple heuristic thresholds: tasks with $w_j \geq w_{\text{RT}}$ route to* `DSQ_PERF`*, long-running batch jobs ($p_{\text{est},j} \geq 3\,\text{ms}$ or* `is_batch`*) route to* `DSQ_EFF`*, and general interactive tasks route to domestic shared queues (*`DSQ_SHARED_P` *or* `DSQ_SHARED_E`*) preserving CPU stickiness. Domain routing does not evaluate $\rho = w/p$ directly.*
2) ***Intra-Queue Scheduling (Algorithm 3):*** *Task density ρ_j and reference density ρ_{ref} are evaluated exclusively within the virtual deadline calculus ($\Delta v_j \propto \rho_{\text{ref}}/\rho_j$) to establish the execution order ($\leq_v$) within shared dispatch queues, prioritizing higher-density tasks ahead of lower-density competitors.*

For interactive tasks ($w_j \geq w_{\text{RT}}$ or $p_j < \tau_{\min}$), $v_j(t) = t$, placing them at the front of the DSQ priority heap. For batch tasks, virtual deadlines advance monotonically in proportion to quantum consumed scaled inversely by execution density ($\Delta v_j \propto 1/\rho_j = p_{\text{est},j}/w_j$), maintaining Generalized Processor Sharing (GPS) fairness while approximating Smith's density ordering.

B. Queue State Dynamics

We model the system across four kernel dispatch queues:

$$\begin{aligned}\mathcal{K} &\triangleq \{1,2,3,4\} \\ &\equiv \{\text{DSQ_PERF, DSQ_SHARED_P,} \\ &\qquad \text{DSQ_SHARED_E, DSQ_EFF}\}\end{aligned} \tag{46}$$

Let $\mathbf{Q}(t) = \big(q_1(t), q_2(t), q_3(t), q_4(t)\big)^T \in \mathbb{Z}^4_{\geq 0}$ represent the queue backlog vector at discrete time slot $t \in \mathbb{Z}_{\geq 0}$. The discrete-time queue state evolution equation is:

$$q_k(t+1) = \max\Big(0,\ q_k(t) - s_k(t)\Big) + a_k(t), \quad \forall k \in \mathcal{K} \tag{47}$$

where:

- $a_k(t)$ is the number of task arrivals into queue k during time slot t, with arrival rate $\lambda_k \triangleq \mathbb{E}[a_k(t)]$ and bounded second moment $\mathbb{E}[a_k(t)^2] \leq a^2_{\max} < \infty$.
- $s_k(t)$ is the service capacity (number of task dispatches plus work-stealing removals) from queue k during time slot t, with $s_k(t) \leq s_{\max} < \infty$.

C. Quadratic Lyapunov Function and Drift Analysis

Definition 4 (Quadratic Lyapunov Function)**.** *We define the quadratic Lyapunov function $L(\mathbf{Q}(t))$ as:*

$$L(\mathbf{Q}(t)) \triangleq \frac{1}{2}\sum_{k=1}^{4} q_k(t)^2 = \frac{1}{2}\|\mathbf{Q}(t)\|_2^2 \tag{48}$$

The conditional one-step Lyapunov drift is defined as:

$$\Delta L(\mathbf{Q}(t)) \triangleq \mathbb{E}\Big[L(\mathbf{Q}(t+1)) - L(\mathbf{Q}(t)) \ \Big|\ \mathbf{Q}(t)\Big] \tag{49}$$

Lemma 2 (Exact Algebraic Lyapunov Drift Upper Bound)**.** *For the queue dynamics in* (47)*, the one-step Lyapunov drift satisfies:*

$$\Delta L(\mathbf{Q}(t)) \leq B - \sum_{k=1}^{4} q_k(t)\,\mathbb{E}\Big[s_k(t) - a_k(t) \ \Big|\ \mathbf{Q}(t)\Big] \tag{50}$$

where B is a finite positive constant defined as:

$$B \triangleq \frac{1}{2}\sum_{k=1}^{4}\mathbb{E}\Big[a_k(t)^2 + s_k(t)^2 \ \Big|\ \mathbf{Q}(t)\Big] \leq 2(a^2_{\max} + s^2_{\max}) < \infty \tag{51}$$

Proof. From the queue evolution equation (47), for each queue k:

$$q_k(t+1) = \max\Big(0,\ q_k(t) - s_k(t)\Big) + a_k(t) \tag{52}$$

Squaring both sides:

$$\begin{aligned} q_k(t+1)^2 &= \Big(\max\Big(0,\ q_k(t) - s_k(t)\Big) + a_k(t)\Big)^2 \\ &= \Big(\max\Big(0,\ q_k(t) - s_k(t)\Big)\Big)^2 + a_k(t)^2 \\ &\quad + 2a_k(t)\max\Big(0,\ q_k(t) - s_k(t)\Big) \end{aligned} \tag{53}$$

We now bound each component of (53):

1) For any real number x, $(\max(0,x))^2 \le x^2$. Setting $x = q_k(t) - s_k(t)$:

$$\Big(\max\Big(0,\ q_k(t) - s_k(t)\Big)\Big)^2 \le \big(q_k(t) - s_k(t)\big)^2 \quad (54)$$

Expanding the square:

$$\big(q_k(t) - s_k(t)\big)^2 = q_k(t)^2 - 2q_k(t)s_k(t) + s_k(t)^2 \quad (55)$$

2) For the cross-term, since $s_k(t) \ge 0$ and $q_k(t) \ge 0$, we have $q_k(t) - s_k(t) \le q_k(t)$, which implies:

$$\max\Big(0,\ q_k(t) - s_k(t)\Big) \le q_k(t) \quad (56)$$

Because $a_k(t) \ge 0$, multiplying both sides by $2a_k(t)$ yields:

$$\begin{aligned} 2a_k(t)\max\Big(0,\ q_k(t) - s_k(t)\Big) &\le 2a_k(t)q_k(t) \\ &= 2q_k(t)a_k(t) \end{aligned} \quad (57)$$

Substituting (54) and (57) back into (53):

$$\begin{aligned} q_k(t+1)^2 &\le q_k(t)^2 - 2q_k(t)s_k(t) + s_k(t)^2 + a_k(t)^2 \\ &\quad + 2q_k(t)a_k(t) \\ &= q_k(t)^2 + a_k(t)^2 + s_k(t)^2 \\ &\quad - 2q_k(t)\big(s_k(t) - a_k(t)\big) \end{aligned} \quad (58)$$

Subtracting $q_k(t)^2$ from both sides and dividing by 2:

$$\begin{aligned} \frac{1}{2}\Big(q_k(t+1)^2 - q_k(t)^2\Big) &\le \frac{1}{2}\big(a_k(t)^2 + s_k(t)^2\big) \\ &\quad - q_k(t)\big(s_k(t) - a_k(t)\big) \end{aligned} \quad (59)$$

Summing over all four queues $k \in \{1,2,3,4\}$ and taking conditional expectations given $\mathbf{Q}(t)$:

$$\begin{aligned} \Delta L(\mathbf{Q}(t)) &\le \sum_{k=1}^{4} \frac{1}{2}\mathbb{E}\big[a_k(t)^2 + s_k(t)^2 \mid \mathbf{Q}(t)\big] \\ &\quad - \sum_{k=1}^{4} q_k(t)\mathbb{E}\big[s_k(t) - a_k(t) \mid \mathbf{Q}(t)\big] \\ &= B - \sum_{k=1}^{4} q_k(t)\mathbb{E}\big[s_k(t) - a_k(t) \mid \mathbf{Q}(t)\big] \end{aligned} \quad (60)$$

Because arrival variances and service capacities per scheduling time slot are physically bounded by finite dispatch limits ($a_k(t) \le a_{\max} < \infty$, $s_k(t) \le s_{\max} < \infty$), the constant $B = \frac{1}{2}\sum_{k=1}^{4}\mathbb{E}[a_k(t)^2 + s_k(t)^2 \mid \mathbf{Q}(t)]$ is strictly finite. □

D. Workload Admissibility, Queue Stability, and Starvation Freedom

Definition 5 (Decoupled Class-Admissible Workload Region)**.** *The arrival workload is class-admissible if:*

1) *Real-time demand to* `DSQ_PERF` *(q_1) is sub-capacity on the Performance partition:*

$$\lambda_1 \bar{p}_1 \le M_P c_P - \epsilon_1, \quad \textit{for some } \epsilon_1 > 0 \quad (61)$$

2) *Aggregate workload demand is strictly within total multiprocessor capacity:*

$$\sum_{k=1}^{4} \lambda_k \bar{p}_k \le C_{\text{total}} - \epsilon_0, \quad \epsilon_0 > 0 \quad (62)$$

where $C_{\text{total}} = M_P c_P + M_E c_E$, $\lambda_k = \mathbb{E}[a_k(t)]$, and $\bar{p}_k$ is the mean nominal execution requirement of tasks arriving in queue k.

Theorem 4 (Conditional Multi-Queue Stability via Foster-Lyapunov Drift)**.** *Let the multi-queue backlog $\mathbf{Q}(t)$ evolve under an aggregate-admissible workload (Definition 5). If the dispatch and hierarchical work-stealing policy guarantees a positive service slack across all active non-empty queues:*

$$\mathbb{E}\big[s_k(t) - a_k(t) \,\big|\, \mathbf{Q}(t)\big] \ge \delta > 0, \quad \forall k : q_k(t) > 0 \quad (63)$$

then the quadratic Lyapunov function satisfies the negative drift condition:

$$\Delta L(\mathbf{Q}(t)) \le B - \delta \sum_{k=1}^{4} q_k(t) \quad (64)$$

for finite positive constant $B = \frac{1}{2}\sum_{k=1}^{4}\mathbb{E}[a_k(t)^2 + s_k(t)^2 \mid \mathbf{Q}(t)] < \infty$. Consequently, the queue backlog process $\mathbf{Q}(t)$ is Foster-Lyapunov stable (positive recurrent), with bounded time-average queue depth:

$$\limsup_{T\to\infty} \frac{1}{T}\sum_{t=0}^{T-1}\sum_{k=1}^{4}\mathbb{E}[q_k(t)] \le \frac{B}{\delta} < \infty \quad (65)$$

Proof. From Lemma 2, the one-step Lyapunov drift satisfies:

$$\Delta L(\mathbf{Q}(t)) \le B - \sum_{k=1}^{4} q_k(t)\,\mathbb{E}\big[s_k(t) - a_k(t) \,\big|\, \mathbf{Q}(t)\big] \quad (66)$$

Under the stated conditional per-queue service slack condition (63), $\mathbb{E}[s_k(t) - a_k(t) \mid \mathbf{Q}(t)] \ge \delta > 0$ for all queues with $q_k(t) > 0$. Substituting this directly into the drift bound yields:

$$\Delta L(\mathbf{Q}(t)) \le B - \sum_{k=1}^{4} q_k(t)\cdot\delta = B - \delta\sum_{k=1}^{4} q_k(t) \quad (67)$$

By the Foster-Lyapunov Stability Criterion (Neveu's Theorem for irreducible Markov chains), the negative drift condition implies that the queue state process $\mathbf{Q}(t)$ is positive recurrent. Summing (64) over time slots $t = 0, 1, \ldots, T-1$:

$$\mathbb{E}[L(\mathbf{Q}(T))] - \mathbb{E}[L(\mathbf{Q}(0))] \le TB - \delta\sum_{t=0}^{T-1}\sum_{k=1}^{4}\mathbb{E}[q_k(t)] \quad (68)$$

Since $L(\mathbf{Q}(T)) \ge 0$:

$$\frac{1}{T}\sum_{t=0}^{T-1}\sum_{k=1}^{4}\mathbb{E}[q_k(t)] \le \frac{B}{\delta} + \frac{\mathbb{E}[L(\mathbf{Q}(0))]}{\delta T} \quad (69)$$

Taking the limit superior as $T \to \infty$ establishes strong aggregate queue stability:

$$\limsup_{T\to\infty} \frac{1}{T}\sum_{t=0}^{T-1}\sum_{k=1}^{4}\mathbb{E}[q_k(t)] \le \frac{B}{\delta} < \infty \quad (70)$$

□

Remark 8 (Architectural Realization of the Service Slack Condition)**.** *In multi-queue systems with strict priorities, global capacity admissibility ($\sum \lambda_k \bar{p}_k \leq C_{\text{total}} - \epsilon_0$) is a necessary condition for aggregate stability, but does not independently ensure per-queue service slack* (63) *if high-priority queues can monopolize all cores. OPTIMA establishes this per-queue condition through structural partitioning: (1) dedicated Efficiency cores provide guaranteed domestic compute capacity to batch queues (`DSQ_EFF`), preventing high-priority P-core traffic from completely starving throughput work; and (2) dynamic quantum scaling $\tau(q_k) = \max(\frac{Q_{\text{base}}}{1+q_k}, \tau_{\min})$ dynamically compresses time slices during congestion, accelerating task dispatch and preemption frequency.*

Proposition 2 (Virtual Deadline Bounding and Starvation Protection)**.** *Under OPTIMA's virtual deadline calculus (Algorithm 3), for every admitted task J_j enqueued into a batch or interactive queue (`DSQ_SHARED_P`, `DSQ_SHARED_E`, `DSQ_EFF`) at time t, its assigned virtual deadline satisfies:*

$$v_j(t) = \text{clamp}(v_{\text{time}}, t, t + 2\tau_j) + \Delta v_j \leq t + 3\tau_{\max} \quad (71)$$

since $\tau_j \leq \tau_{\max}$ and $\Delta v_j \leq \tau_{\max}$. Any subsequent batch task $J_{j'}$ arriving at timestamp $t' > t + 3\tau_{\max}$ receives $v_{j'}(t') \geq t' > v_j(t)$ and cannot be scheduled ahead of J_j. While a batch task arriving earlier ($t' \leq t + 3\tau_{\max}$) with higher density may receive an earlier deadline, $v_j(t) \leq t + 3\tau_{\max}$ strictly bounds relative queue advancement lag. Under the conditional service rate condition ($\mathbb{E}[s_k - a_k \mid \mathbf{Q}] \geq \delta > 0$) of Theorem 4, every admitted batch task is guaranteed dispatch within finite time. Furthermore, latency-critical tasks ($w \geq w_{\text{RT}}$) receive immediate priority ($v_j = t$) and always supersede batch tasks; when preemption is suppressed by OAPF, the immediate waiting delay imposed by the running task is bounded by $\tau_{\max}$, ensuring bounded priority inversion.

V. Algorithmic Specifications

We formalize OPTIMA's core scheduling operations implemented in Linux `sched_ext` eBPF: CPU selection, queue insertion, and work-conserving dispatch.

References

[1] W. E. Smith, "Various optimizers for single-stage production," *Naval Research Logistics Quarterly*, vol. 3, no. 1–2, pp. 59–66, 1956.

[2] T. Kawaguchi and S. Kyan, "Worst case bound of an LRF schedule for parallel processors," *SIAM Journal on Computing*, vol. 15, no. 4, pp. 1119–1129, 1986.

[3] J. K. Lenstra, A. H. G. Rinnooy Kan, and P. Brucker, "Complexity of machine scheduling problems," *Annals of Discrete Mathematics*, vol. 1, pp. 343–362, 1977.

[4] J. Bruno, E. G. Coffman, and R. Sethi, "Sequencing problems with partially ordered tasks and alternative processors," *Journal of the ACM*, vol. 21, no. 4, pp. 667–676, 1974.

[5] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco, CA: W. H. Freeman and Company, 1979.

[6] M. J. Neely, *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Morgan & Claypool Publishers, 2010.

[7] L. Georgiadis, M. J. Neely, and L. Tassiulas, "Resource allocation and cross-layer control in wireless networks," *Foundations and Trends in Networking*, vol. 1, no. 1, pp. 1–144, 2006.

[8] I. Stoica and H. Abdel-Wahab, "Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation," Old Dominion University, Tech. Rep. TR-95-22, 1995.

[9] T. Heo, D. Vernet, A. Righi, and C. Min, "sched_ext: BPF Extensible CPU Scheduling in the Linux Kernel," in *Proceedings of the Linux Plumbers Conference (LPC)*, 2024.

[10] N. Bansal, H. L. Chan, T. W. Lam, and K. K. To, "Strictly online scheduling with speed augmentation," *Theoretical Computer Science*, vol. 410, no. 47, pp. 4963–4973, 2009.

Algorithm 2 OPTIMA-SELECT-CPU (Synchronous IPC Fast-Path & Asymmetric Domain Selection)

Require: Task p, previous CPU $prev_cpu$, wakeup flags $wake_flags$
Ensure: Selected target CPU $target_cpu$

$tctx \leftarrow$ lookup_task_ctx(p)
if p is pinned OR p is kernel thread **then**
    Dispatch directly to $prev_cpu$ via `SCX_DSQ_LOCAL`; **return** $prev_cpu$
end if
$this_cpu \leftarrow$ smp_processor_id(), $idle_smtmask \leftarrow$ scx_bpf_get_idle_smtmask(), $prev_is_perf \leftarrow$ is_perf_core($prev_cpu$)
if $wake_flags$ & SCX_WAKE_SYNC **then**
    if $this_cpu \in p$.cpus_allowed **and** dsq_nr_queued($this_cpu$) $== 0$ **then**
        goto direct_dispatch with $target_cpu \leftarrow this_cpu$
    end if
    $this_sib \leftarrow$ get_smt_sibling($this_cpu$)
    if $this_sib \geq 0$ **and** $this_sib \in p$.cpus_allowed **and** dsq_nr_queued($this_sib$) $== 0$ **then**
        test_and_clear_idle($this_sib$); **goto** direct_dispatch with $target_cpu \leftarrow this_sib$
    end if
    if $prev_cpu \in p$.cpus_allowed **and** test_and_clear_idle($prev_cpu$) **then**
        goto direct_dispatch with $target_cpu \leftarrow prev_cpu$
    end if
end if
if $prev_is_perf$ **and** $prev_cpu \in p$.cpus_allowed **and** $prev_cpu \in idle_smtmask$ **and** test_and_clear_idle($prev_cpu$) **then**
    goto direct_dispatch with $target_cpu \leftarrow prev_cpu$
end if
if $tctx.core_type \neq$ CORE_TYPE_EFF **then**
    $c \leftarrow$ pick_idle_domestic_core(p, is_perf $=$ TRUE, $idle_smtmask$)
    if $c \geq 0$ **then**
        goto direct_dispatch with $target_cpu \leftarrow c$
    end if
    $c \leftarrow$ pick_idle_domestic_cpu(p, is_perf $=$ TRUE)
    if $c \geq 0$ **then**
        goto direct_dispatch with $target_cpu \leftarrow c$
    end if
end if
if $prev_is_perf$ **and** $p \in p$.cpus_allowed **then**
    if dsq_nr_queued(DSQ_SHARED_P) > 0 **or** ($tctx.is_batch \wedge tctx.avg_runtime \geq$ 3ms) **then**
        $e \leftarrow$ pick_idle_domestic_core(p, is_perf $=$ FALSE, $idle_smtmask$)
        if $e \geq 0$ **then**
            goto direct_dispatch with $target_cpu \leftarrow e$
        end if
    end if
    $tctx.dispatch_local \leftarrow$ FALSE; **return** $prev_cpu$
end if
$e \leftarrow$ pick_idle_domestic_core(p, is_perf $=$ FALSE, $idle_smtmask$)
if $e \geq 0$ **then**
    goto direct_dispatch with $target_cpu \leftarrow e$
end if
if $tctx.weight \geq w_{\mathrm{RT}}$ **then**
    $preempt_cpu \leftarrow$ OPTIMA-OAPF($p, tctx, \mathcal{P}$)
    if $preempt_cpu \geq 0$ **then**
        $tctx.dispatch_local \leftarrow$ TRUE; **return** $preempt_cpu$
    end if
end if
$tctx.dispatch_local \leftarrow$ FALSE; **return** $prev_cpu$
direct_dispatch:
Compute slice $\tau \in [\tau_{\min}, \tau_{\max}]$ based on $tctx.is_batch$ and $tctx.avg_runtime$
scx_bpf_dsq_insert(p, SCX_DSQ_LOCAL, τ, 0)
return $target_cpu$

Algorithm 3 OPTIMA-ENQUEUE (Virtual Deadline Calculus & OAPF Preemption Kick)

Require: Task p, enqueue flags enq_flags
$tctx \leftarrow$ lookup_task_ctx(p)
$task_cpu \leftarrow$ scx_bpf_task_cpu(p)
if p is pinned OR p is kernel thread OR $tctx.dispatch_local$ **then**
  $tctx.dispatch_local \leftarrow$ FALSE
  scx_bpf_dsq_insert(p, SCX_DSQ_LOCAL, $\tau_{\min}$, enq_flags); **return**
end if
{Density-Guided Domain Placement Target DSQ Selection}
if $tctx.weight \geq w_{\mathrm{RT}}$ **then**
  $target_dsq \leftarrow$ DSQ_PERF
else if $tctx.avg_runtime \geq$ 3ms $\vee$ $tctx.is_batch$ **then**
  $target_dsq \leftarrow$ DSQ_EFF
else
  $target_dsq \leftarrow$ is_perf_core($task_cpu$) ? DSQ_SHARED_P : DSQ_SHARED_E
end if
$is_rt \leftarrow (tctx.weight \geq w_{\mathrm{RT}} \vee (\neg tctx.is_batch \wedge tctx.avg_runtime < 1\text{ms}))$
$now \leftarrow$ scx_bpf_now()
if is_rt **then**
  $\tau \leftarrow \tau_{\min}, \quad deadline \leftarrow now$
else
  if $tctx.avg_runtime \geq$ 3ms **then**
    $\tau \leftarrow \tau_{\max}$
  else
    $q \leftarrow$ scx_bpf_dsq_nr_queued($target_dsq$)
    $\tau \leftarrow \max\left(\frac{Q_{\mathrm{base}}}{1+q},\ \tau_{\min}\right)$
  end if
  $v \leftarrow \mathrm{clamp}(p.\text{scx.dsq_vtime}, now, now + 2\tau)$
  $p_{\mathrm{est}} \leftarrow \max(tctx.avg_runtime, \tau_{\min})$
  $\rho \leftarrow \frac{\max(tctx.weight, 10)}{p_{\mathrm{est}}}$ {Execution density $\rho = w/p_{\mathrm{est}}$}
  $\Delta v \leftarrow \mathrm{clamp}\left(\frac{\tau \cdot p_{\mathrm{est}}}{10 \cdot \max(tctx.weight, 10)},\ \tau_{\min},\ \tau_{\max}\right)$ {Monotonic increment $\tau \cdot \rho_{\mathrm{ref}}/\rho$}
  $deadline \leftarrow v + \Delta v$
end if
p.scx.dsq_vtime $\leftarrow deadline$
scx_bpf_dsq_insert_vtime($p, target_dsq, \tau, deadline, enq_flags$)
{Online Admissible Preemption Kick}
$idle_cpu \leftarrow$ scx_bpf_pick_idle_cpu(p.cpus_allowed, 0)
if $idle_cpu \geq 0$ **then**
  scx_bpf_kick_cpu($idle_cpu$, SCX_KICK_IDLE); **return**
end if
if is_rt **then**
  $preempt_cpu \leftarrow$ OPTIMA-OAPF($p, tctx, \mathcal{P}$)
  if $preempt_cpu \geq 0$ **then**
    scx_bpf_kick_cpu($preempt_cpu$, SCX_KICK_PREEMPT)
  end if
end if

Algorithm 4 OPTIMA-DISPATCH (Asymmetric Hierarchical Work-Stealing)

Require: Executing core cpu, previous task $prev$
$is_perf \leftarrow$ is_perf_core(cpu)
if is_perf **then**
  {Priority Tier 1: Latency-critical tasks}
  if scx_bpf_dsq_move_to_local(DSQ_PERF, 0) **then**
    return
  end if
  {Priority Tier 2: Domestic P-core shared queue}
  if scx_bpf_dsq_move_to_local(DSQ_SHARED_P, 0) **then**
    return
  end if
  {Priority Tier 3: Cross-domain work-stealing from E-domain}
  if scx_bpf_dsq_move_to_local(DSQ_SHARED_E, 0) **then**
    return
  end if
  {Priority Tier 4: Throughput steal from batch queue}
  if scx_bpf_dsq_move_to_local(DSQ_EFF, 0) **then**
    return
  end if
else
  {Priority Tier 1: Domestic E-core shared queue}
  if scx_bpf_dsq_move_to_local(DSQ_SHARED_E, 0) **then**
    return
  end if
  {Priority Tier 2: Throughput batch queue}
  if scx_bpf_dsq_move_to_local(DSQ_EFF, 0) **then**
    return
  end if
  {Priority Tier 3: Cross-domain steal from P-domain}
  if scx_bpf_dsq_move_to_local(DSQ_SHARED_P, 0) **then**
    return
  end if
  {Priority Tier 4: Work-conserving safety steal from RT queue}
  if scx_bpf_dsq_move_to_local(DSQ_PERF, 0) **then**
    return
  end if
end if